

Aetheric Geometry

A gesture- and voice-controlled additive instrument:
design, signal processing, and measured corrections

Lluís Estapé

Universitat Politècnica de Catalunya

lluis.estape@estudiantat.upc.edu

Abstract

Aetheric Geometry is a real-time instrument played with bare hands in front of a webcam. Hand landmarks are reduced to a polygon whose geometry drives an additive synthesiser tuned in just intonation, and a bilingual offline speech layer freezes a sound and lets its effects be adjusted by turning a hand as one would turn a knob. This report documents the system as built, but its subject is narrower than that: it is an account of what measurement found in code that already worked. A phase accumulator that looked correct was detuning every note by -3.385 cents and emitting a glitch at 86.13 Hz; a reverberator whose four comb filters shared one feedback gain decayed at four different rates; a waveform morph collapsed by 14 dB in the middle of its travel through phase cancellation; and a speech layer with a relaxed threshold fired six mutually contradictory commands from one sentence. Each is derived analytically, measured, corrected, and re-measured. The instrument runs its audio callback at 10.5% of the deadline and its interface at 7.5 ms per frame.

1 Introduction

The instrument occupies a lineage that predates its technology: instruments played by moving the body through a field rather than by touching a key. What the camera changes is not the idea but its cost. A webcam and a hand-landmark model give twenty-one three-dimensional points per hand at video rate, and the design question becomes what to do with them.

Aetheric Geometry answers it with a deliberately literal mapping. Two hands pinch to open a *thread* whose length sets pitch; pinching further fingers adds vertices to a *polygon* whose centroid, area, and aspect ratio drive pitch, filter, reverberation, and tremolo. The polygon is not a metaphor. It is a figure with measurable properties, and each property is bound to one parameter, so the player can predict the sound from the shape.

Two additions turn this from a demonstration into something playable. The first is *hold*: a spoken word freezes the sounding chord, releasing the hands from the job of maintaining it, after which they operate a virtual rotary control. The second is a bilingual offline voice layer that accepts either isolated commands or whole sentences. Both are described in Sections 5 and 6.

The system runs entirely offline: no network, no cloud speech, no external audio engine. It ships as a Windows desktop application.

2 How the project began, and what changed

The instrument existed before this work in a form that was already complete as a piece of interaction design. The gesture pipeline, the state machine, the polygon mapping, and the visual language were all in place and behaved as intended. What it lacked was any evidence that its signal processing did what its own source comments claimed.

That distinction matters because none of the defects reported here announced themselves. The instrument made sound; the sound was recognisably the sound intended. The faults were found by writing down what each block was supposed to compute, computing it, and comparing:

- The oscillator bank was detuned by a constant -3.385 cents and emitted a periodic discontinuity at 86.13 Hz (Sec. 4.1).
- Sawtooth and square waves were generated without band limiting, aliasing at -12 dB relative to signal at 2 kHz (Sec. 4.2).
- The reverberator's four comb filters shared a single feedback gain, giving a 13.7% spread in decay time, and had no diffusion stage (Sec. 4.4).
- Envelope times were stored as raw one-pole coefficients, so every envelope changed with the device sample rate.
- Just-intonation ratios were stored as rounded decimals (Sec. 3.2).
- MIDI note numbers were emitted an octave flat.

The work also added the hold-and-knob layer, the voice layer, a continuous waveform morph, MIDI output, a rebuilt interface, and a test suite that now stands at 195 cases. Table 1 summarises the resulting structure.

3 Mapping geometry to sound

3.1 Pitch

Pitch perception is logarithmic in frequency, so a linear map from hand position would leave the low end musically cramped. The control value $t \in [0, 1]$ is mapped exponentially,

$$f(t) = f_{lo} \left(\frac{f_{hi}}{f_{lo}} \right)^t, \quad (1)$$

Module	Lines	Responsibility
<code>dsp.py</code>	290	Oscillators, filters, delay lines
<code>synth.py</code>	421	Engine, voicing, hold
<code>tuning.py</code>	124	Exact ratios, pitch mapping
<code>knob.py</code>	221	Rotary control geometry
<code>voice.py</code>	377	Bilingual recognition, triage
<code>vocabulary.py</code>	217	Per-language word tables
<code>ui.py</code>	425	Typography, panels, compositing
<code>renderer.py</code>	451	Gesture overlay and HUD
<code>midiout.py</code>	191	CC / pitch-bend output
<code>main.py</code>	450	Event loop, state machine
<code>tests/</code>	1190	195 cases

Table 1: Module structure. `dsp`, `tuning`, `knob` and `vocabulary` have no camera, audio, or model dependencies, which is what makes the numeric core testable and measurable without hardware.

which gives constant cents per unit of hand travel. The result is then snapped to the nearest degree of an A minor pentatonic scale, so that a moving hand produces a playable line rather than a continuous glide.

3.2 Just intonation, and its limit

Chords are built from exact rational ratios above the root, stored as `Fraction` rather than as decimals. This is not pedantry. The point of just intonation is that partials of different voices coincide *exactly*, so that beating between them vanishes, and rounding destroys precisely that property. The perfect fourth had been stored as 1.333; against the true $4/3$ the error is

$$1200 \log_2 \left(\frac{1.333}{4/3} \right) = -0.433 \text{ cents}, \quad (2)$$

inaudible as pitch, but not as beating: a fourth above 220 Hz is 293.333 Hz exact and 293.260 Hz rounded, and the third partial of the root then beats against the second partial of the fourth at roughly 0.15 Hz — a slow wow across a sustained chord, which is the artefact just intonation exists to remove.

The system is honest about its own boundary. These are fixed ratios above a *movable* root, and the root is quantised to equal temperament. The instrument is therefore justly tuned within a chord and equal-tempered between chords. A fully just system has no single answer for what happens when the root moves, because stacking exact fifths never closes the octave: twelve of them overshoot by the Pythagorean comma,

$$1200 \log_2 \frac{(3/2)^{12}}{2^7} = 23.460 \text{ cents}. \quad (3)$$

Choosing a fixed ratio set over a tempered root avoids comma drift at the cost of transposition purity.

4 The synthesis engine

4.1 A phase accumulator that was one sample short

The oscillator rendered a block by forming a phase ramp and then storing the phase back for the next block. It stored the phase of the *last sample rendered* rather than the phase the next block should *begin* on.

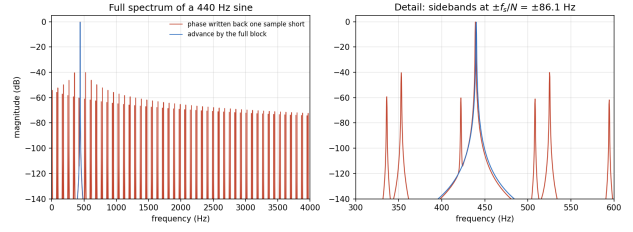


Figure 1: A 440 Hz sine rendered by both accumulators. Left: full spectrum. Right: detail around the carrier, showing the sidebands at $\pm f_s/N = \pm 86.1$ Hz produced by replaying one sample of phase per block.

With a block of N samples the oscillator therefore advanced $N - 1$ increments where it owed N .

Two consequences follow. The effective frequency is scaled by $(N - 1)/N$, a constant detuning of

$$1200 \log_2 \frac{N - 1}{N} = -3.385 \text{ cents} \quad (N = 512), \quad (4)$$

and one sample of phase is replayed at every block boundary, producing a discontinuity repeating at $f_s/N = 86.13$ Hz. Measurement confirms both: the nominal 440 Hz tone appears at 439.08 Hz before the fix and 440.09 Hz after (bin resolution 0.34 Hz), and sidebands appear at ± 86 Hz around the carrier.

The irony is worth stating. In an instrument built around exact frequency ratios, every note was globally 3.4 cents flat with an 86 Hz buzz on top — and because the error scales all partials equally, the ratios themselves survived, which is exactly why nothing sounded obviously wrong.

4.2 Band limiting

A trivially generated sawtooth has harmonics at every multiple of f_0 with amplitude $1/k$; all those above $f_s/2$ fold back as inharmonic partials. In an instrument whose pitch is a continuous function of hand position the aliases slide as the hand moves, which is far more noticeable than static aliasing on a fixed note.

PolyBLEP replaces the discontinuity with a two-sample polynomial approximation of a band-limited step. Writing ϕ for normalised phase and $\Delta = f_0/f_s$, the residual subtracted at the wrap is

$$r(\phi) = \begin{cases} 2u - u^2 - 1, & u = \phi/\Delta, \phi < \Delta, \\ u^2 + 2u + 1, & u = (\phi - 1)/\Delta, \phi > 1 - \Delta, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

The square wave applies the same residual at both edges; the triangle is obtained by integrating the band-limited square with a leaky integrator whose pole sits at $e^{-2\pi f_{\text{leak}}/f_s}$.

Figure 2 shows the result. The correction buys a consistent ≈ 16 dB: at 2031 Hz the sawtooth improves from -12.1 dB to -27.2 dB, and the square from -13.8 dB to -28.0 dB.

Measuring this required one methodological decision that carried more weight than the correction itself. Each test fundamental is snapped to an exact FFT bin, so that all harmonics fall on bins. With no spectral

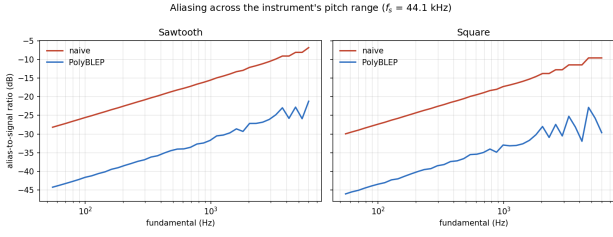


Figure 2: Alias-to-signal ratio across the instrument’s pitch range. Each fundamental is placed on an exact FFT bin, so every true harmonic lands on a bin and any off-grid energy is aliasing — no window is needed and none is applied.

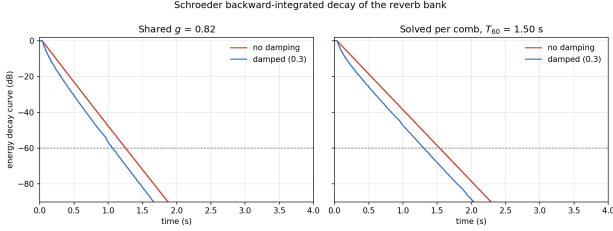


Figure 3: Schroeder backward-integrated energy decay. Left: one shared feedback gain. Right: per-comb gains solved from a single T_{60} target.

leakage, energy off the harmonic grid *is* aliasing, with no windowing assumption to defend.

4.3 A prewarped filter

The lowpass had used the naive one-pole $c = e^{-2\pi f_c/f_s}$, which is accurate only while $f_c \ll f_s$. The topology-preserving-transform form uses $g = \tan(\pi f_c/f_s)$ and $G = g/(1 + g)$, collapsing to

$$y[n] = Gx[n] + Gx[n-1] + (1 - 2G)y[n-1], \quad (6)$$

which is the bilinear transform of $1/(1 + s/\omega_c)$ with frequency prewarping. Unity gain at DC and an exact zero at Nyquist follow by substituting $z = 1$ and $z = -1$. Measured, the TPT form sits at -3.010 dB at every requested cutoff from 100 Hz to 20 kHz; the naive form drifts to -0.987 dB at 20 kHz, a 2 dB error.

4.4 A reverberator that decayed four ways

The reverberator is a bank of four parallel feedback combs followed by allpass diffusers. A comb of delay D and gain g emits echoes every D/f_s seconds, the n -th attenuated by g^n . Reaching -60 dB requires $g^n = 10^{-3}$, hence

$$T_{60} = \frac{-3D}{f_s \log_{10} g} \iff g = 10^{-3D/(f_s T_{60})}. \quad (7)$$

The original code gave all four combs the same $g = 0.82$. By Eq. (7) that makes T_{60} proportional to D , and with delays $\{1557, 1617, 1491, 1422\}$ the bank decays over 1.122–1.276 s: a 13.7% spread, heard as a tail that comes apart rather than fading as one object. Solving the right-hand form of Eq. (7) per comb from a single T_{60} target ties them together; measured decay then matches the target to three decimal places.

Two further corrections were made. Allpass diffusers

of the canonical single-delay-line form

$$H(z) = \frac{-g + z^{-D}}{1 - gz^{-D}}, \quad |H(e^{j\omega})| = 1, \quad (8)$$

were added after the comb bank: the combs set the decay, the allpasses set echo density. And a one-pole lowpass was placed inside each comb’s feedback path, making T_{60} frequency-dependent so that high partials die first, as they do in a room.

4.5 Continuous waveform morph

The four waveforms are the integer steps of one continuous control, ordered by harmonic content — sine, triangle, sawtooth, square — so that the knob behaves as a single edge control. Intermediate positions cross-fade the two neighbours; both are band limited and share a phase accumulator, so their sum is band limited too.

The first implementation was unusable, and the reason is instructive. Measured across the sweep, the sawtooth-to-square segment collapsed to an RMS of 0.188 against endpoints of 0.574 and 0.995 — roughly 14 dB down, audible as the volume dropping out mid-travel. The cause is phase: a rising ramp is negative through the first half of a cycle while a square is positive through it, so mixing them subtracts.

The fix is to align the fundamental phase of all four shapes, which is invisible on any shape alone — inverting a sawtooth or shifting a triangle by a quarter cycle is inaudible in isolation — but is the difference between a morph that works and one that does not. After alignment the sweep is monotone in character and spans 0.54 to 1.00 RMS, which is the genuine loudness difference between a triangle and a square rather than a defect.

5 Hold and the rotary hand control

Saying *hold* freezes the sounding chord. The engine’s `set_params` becomes a no-op while held — that is what held *means*, the hands stop driving pitch and level — while `set_effect` continues to apply.

The freed hands then operate a rotary control. Pinching grips it; rotating the hand in the camera plane turns it; releasing the pinch lets go while retaining the value, so the arm can be repositioned and the turn continued. Rotation accumulates, giving unlimited travel.

Two design decisions came from discarding something that did not survive contact with a webcam. An earlier version used a flat palm facing the ceiling, moved vertically; a palm facing the ceiling is nearly edge-on to the lens, its landmarks are badly conditioned, and the grip flickered. Rotation about the camera axis is the opposite: it is the most visible thing a hand can do, and the wrist-to-knuckle vector that measures it is long and well separated from the noise floor.

Second, finger extension is measured radially — tip-to-wrist distance divided by palm width — rather than by comparing a fingertip’s y against its knuckle’s. The vertical test assumes an upright hand, and this gesture *rotates* the hand, which is precisely when that assumption fails. Normalising by palm width also makes the measure invariant to distance from the camera.

Angles are unwrapped to $[-\pi, \pi]$ before accumulation. This imposes a sampling limit worth naming: a rotation of more than half a revolution between two video frames is indistinguishable from a smaller one in the opposite direction. This is the Nyquist criterion applied to angle; at 30 fps it allows 15 revolutions per second and never binds in practice.

6 Voice control

Speech recognition is offline (Vosk), with English and Spanish models running *in parallel* on one microphone stream. This is not a preference. An acoustic model is language-specific: *hold* is absent from the Spanish lexicon and *congela* from the English one, and a constrained grammar may only contain words its model already knows. Decoding twice is the only way to hear both.

The layer performs keyword spotting, not parsing. A result is scanned word by word and anything in the vocabulary fires; *vamos a probar una onda cuadrada* works because *cuadrada* is in it, not because the sentence was understood. Ordinary sentence glue — articles, pronouns, verbs of request — is included in the grammar but mapped to nothing, because a closed grammar is closed: every sound reaching it lands on *some* entry, and without somewhere harmless to put the other five words of a sentence they smear across the words that are commands.

6.1 Four gates, each from a measurement

Table 2 lists them. Two deserve explanation.

Level, not confidence. A constrained grammar assigns room tone to its nearest listed word, sometimes at high confidence: 45 s of silence produced a command at confidence 0.87. Confidence answers *which word is this most like*, never *was anyone speaking*. Speech into a laptop microphone runs an order of magnitude above room tone, so peak RMS answers the second question cleanly.

Length triage. Requiring a wake word for everything makes the instrument unusable — short commands must work mid-performance with no ceremony — while requiring nothing lets room conversation play the instrument. Length decides: utterances of three words or fewer fire bare; longer ones require the wake word *aether* and are otherwise dropped whole. Measured, ordinary conversation produced 11 spurious commands in 40 s without triage and none with it.

Gate	Question	Without it
Final results only	utterance done?	11 cmds / 25 s
Peak RMS level	did anyone speak?	1 cmd / 45 s
Length triage	meant for us?	11 cmds / 40 s
One per family	which intent?	6 per sentence

Table 2: Each gate answers a different question, and each was added after measuring the failure that motivated it. Counts are spurious commands observed in ordinary room conditions with nobody addressing the instrument.

6.2 One intent per family

The last gate is the subtlest. Inside a phrase that followed the wake word, per-word confidence is judged more leniently, because connected speech scores far lower than an isolated word and a single strict threshold accepted *eco* alone while silently dropping the identical word inside a sentence — which reads, from outside, as the wake word working and the sentence being ignored.

That leniency then produced a new failure. A single utterance was logged emitting `wave:SIN`, `release`, `hold`, `select:reverb`, `wave:SQUARE` and `wave:TRIANGLE` within four milliseconds. The last one wins and the instrument appears to ignore the request.

The correction groups candidates by intent family — a selection, a waveform, a relative move, a hold — and fires the best-scoring word in each. Words within a family are alternatives; families are independent, so *aether*, *take the echo and lower it thirty percent* legitimately means two things and does both. Mutually exclusive families (hold against release) resolve to the higher confidence.

Spoken numbers are read as percentages, since Vosk emits words rather than digits and *treinta por ciento* must be looked up rather than parsed.

7 Real-time behaviour

7.1 Audio

A block of 512 frames at 44.1 kHz must be produced within 11.61 ms. Measured over 400 blocks, the worst case — eight partials, triangle waveform, every effect engaged — has a 95th-percentile cost of 1.22 ms, or **10.5 %** of the deadline.

That headroom is the product of one structural observation. A feedback comb $y[n] = x[n] + g y[n-D]$ looks like a sequential recursion, but when D is at least the processing chunk length, every sample it reads was written before the current chunk began. The recursion then unrolls into a delay-line read and a vector multiply. Splitting each block at the delay-line wrap guarantees that condition, which is what allows four combs and two allpasses to run in a Python callback without a per-sample loop.

7.2 Interface

The overlay is drawn with real typefaces through Pillow rather than OpenCV’s Hershey fonts. Rendered naively it cost 39 ms per frame. Three changes brought that to 4.6–7.5 ms:

- Rasterised tiles are cached against their content, so most frames become a composite rather than a redraw.
- Channel order and float promotion are done once at rasterise time rather than per frame; the composite alone fell from 5.9 ms to 1.4 ms.
- The effects panel is split into three tiles by how fast each changes, so turning a control redraws a dial rather than a panel.

One defect here is worth recording because it was invisible: the effect-meter loop used `key` as its loop

variable, shadowing the cache key computed above it. Tiles were stored under the wrong key and the cache never hit. The panel looked correct and simply ran at full cost indefinitely.

Camera frames are resampled to the display size *before* the overlay is drawn. Many webcams deliver only 1920×1080 and ignore requests for anything smaller, and letting the window rescale the finished frame resampled rendered text by an arbitrary ratio — the operation that makes type look chewed, and one no amount of better typography survives.

8 Output and packaging

Effect values, pitch, and note state are mirrored as MIDI, which turns the whole gesture and voice surface into a general-purpose controller: the internal engine becomes the demonstration and the control surface becomes the product. Values are quantised to the 0–127 that a control change actually carries and compared against the last value sent; pitch is sent as one held note plus pitch bend rather than as a note per semitone, because retriggering per semitone on a continuous pitch would sound like a machine gun.

Two properties were measured through a loopback port (`analysis/measure_midi.py`). **Transport delay**, timed by opening the same virtual cable as both output and input and pairing each message with its arrival, has a median of 5.65 ms (min 5.01, p95 6.32, 200/200 delivered). Timing `send()` alone would have measured the call returning rather than the message arriving, which is a different and much smaller number. That figure is an upper bound rather than pure driver cost, since it includes Python thread scheduling on the receiving side — but it is the bound that matters, because it sits on a pipeline whose dominant term is the 33 ms video frame period. The transport is not what the player feels.

Traffic confirms the send-on-change design rather than asserting it. A still hand held for 120 frames emitted **zero** messages. Six controls swept simultaneously — far beyond what one pair of hands can do — emitted 163 messages per second, or 15.7% of the roughly 1041 three-byte messages per second a MIDI 1.0 stream carries. The rate limiter therefore never binds in practice, and the automation a DAW records is continuous rather than stepped.

8.1 Reaching a host

Windows provides no virtual MIDI port of its own, so the loop is closed with loopMIDI. Output was confirmed arriving in FL Studio, which plays notes sent down the cable through whichever generator is selected.

Two practical findings are worth recording because both cost time and neither is documented anywhere obvious.

First, **the same virtual cable is not called the same thing on both sides**. The `rtmidi` backend appends each port’s ordinal *within its own direction’s list*, so the output `loopMIDI Port 1` is the input `loopMIDI Port 0`. Matching a loopback pair on the full string finds nothing; the pairing has to be done on the name

with its trailing index stripped. This is exactly the kind of detail that makes a working cable look broken.

Second, **a host’s MIDI-learn latches onto the first control change it sees**. Running the whole instrument to generate one moves six controls at once, so the mapping lands on whichever message happened to arrive first — rarely the intended one, and tedious to undo. A dedicated tool (`analysis/midi_learn.py`) therefore sends a single CC on a slow triangular sweep across the full 0–127 range: one control removes the ambiguity, and the sweep gives the host a range to infer rather than a single value it may miss between window redraws.

Diagnosis follows the same principle. `analysis/midi_ping.py` sends notes rather than control changes, on every loopback port in turn. Verifying a cable with a control change requires finding an activity indicator in the host and trusting it is the right one; notes need no indicator, because if the host is receiving, the instrument plays and the result arrives through the ear. Walking every port also separates *is the cable working* from *is the host listening to this particular cable*, two questions a single-port test conflates into one unhelpful silence.

The application is frozen with PyInstaller into a one-folder Windows build. Two constraints shaped it. The bundle carries two speech models and MediaPipe’s graph data — roughly 150 MB — so a single-file build would re-extract that on every launch. And `matplotlib` cannot be excluded despite nothing in the application plotting anything, because MediaPipe’s drawing utilities import it at module scope; excluding it to save 40 MB produced a build that died on `import mediapipe` behind a dialog reading only *Unhandled exception in script*.

9 Limitations

Gesture and voice contend for three parameters. While not held, the polygon mapping rewrites filter, reverberation, and tremolo every frame, so a spoken adjustment to those three is undone on the next video frame. The remaining parameters are unaffected because no gesture writes them. The fix is parameter ownership — a value touched by voice should resist the gesture path until released — and is not yet implemented.

The main loop is untested. Every numeric module has unit coverage, but the event loop, state machine, and rendering call sequence live in a single 450-line function with none. Extracting the state machine into a class would make the core of the program testable.

Typography is substituted. The interface specifies Inter and IBM Plex Mono; neither ships with Windows and a desktop build has nowhere to fetch a webfont. The nearest installed equivalents are used instead.

The end-to-end chain is not yet measured. What has been timed is the transport — 5.65 ms median from send to arrival — and separately the video and audio stages. What has not is the whole path from a hand moving to a mapped parameter audibly changing

in a host, which is the only figure a producer would care about. It is a harder measurement than it sounds, because the endpoint is acoustic: it needs the DAW’s output recorded alongside a timestamped gesture, with the host’s own buffer and any parameter smoothing included in the total. Until that exists, the honest statement is that each stage is small and the sum has not been demonstrated.

10 Conclusion

The instrument works, and worked before this study began. What the study adds is the difference between an instrument that sounds right and one that can be shown to be right. Every correction reported here was found the same way: by writing down what a block was supposed to compute, computing it, and comparing — and in every case the defect had been inaudible as a defect, presenting instead as a slight dullness, a tail that came apart, a control that felt uneven, or a voice command that seemed not to register.

The measurements are reproducible from the repository: `analysis/measure_dsp.py` regenerates every figure and number in Sections 4 and 5, `analysis/measure_midi.py` times the output transport and counts its traffic, and `analysis/voice_monitor.py` exposes the speech layer’s decisions word by word with confidence and level.

References

1. M. R. Schroeder, “Natural sounding artificial reverberation,” *J. Audio Eng. Soc.*, vol. 10, no. 3, 1962.
2. V. Välimäki and A. Huovilainen, “Antialiasing oscillators in subtractive synthesis,” *IEEE Signal Processing Magazine*, vol. 24, no. 2, 2007.
3. V. Zavalishin, *The Art of VA Filter Design*, rev. 2.1.0, 2018.
4. T. Stilson and J. O. Smith, “Alias-free digital synthesis of classic analog waveforms,” *Proc. ICMC*, 1996.
5. M. Wright and D. Wessel, “Problems and prospects for intimate musical control of computers,” *Computer Music Journal*, vol. 26, no. 3, 2002.